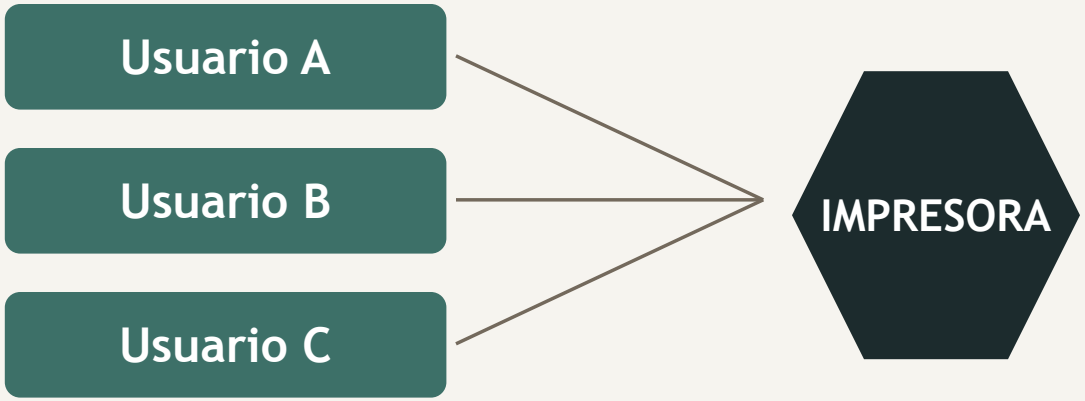
Colas, Árboles y Grafos

Programador Universitario de Sistemas
1° Año - Primer Cuatrimestre

¿Qué pasa si el primero en llegar debe salir primero?

Pensemos en una impresora compartida por varios usuarios.



A llega primero, B después, C último. La impresora debería atender en ese mismo orden: A → B → C.

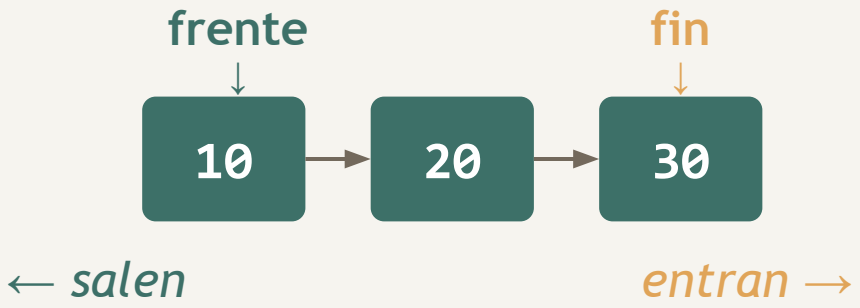
No necesitamos LIFO (el último que llegó no debería imprimir primero sólo porque llegó después). Necesitamos una regla distinta.

FIFO — First In, First Out

La pila restringía el acceso a un extremo con regla LIFO. La cola también restringe a los extremos, pero con la regla opuesta: FIFO.

Una cola: dos extremos, regla FIFO

Una cola permite agregar por un extremo (el fin) y retirar por el otro (el frente).



Cola vs. Pila

PILA → LIFO

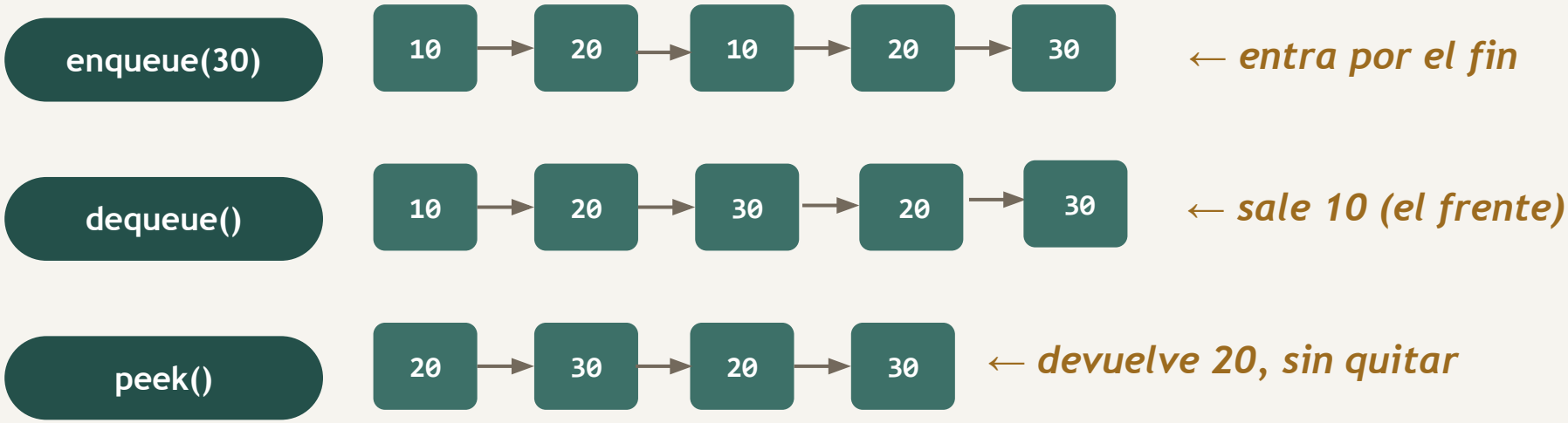
COLA → FIFO

La diferencia no está en cómo se guardan los datos — está en qué elemento sale primero.

Las operaciones básicas son enqueue (agregar), dequeue (quitar y devolver el frente) y peek (consultar el frente).

Las operaciones de una cola, en una sola vista

Para no repetir tres veces el mismo tipo de diagrama, veamos las tres operaciones juntas, con la misma cola de partida.



isEmpty() —
una cola también puede empezar vacía (frente = fin = null). No se puede hacer dequeue() sin elementos: hay que consultar isEmpty() antes.

Implementación con nodos: frente y fin

Reutilizamos el mismo Nodo de la clase anterior. La cola, a diferencia de la pila, necesita guardar DOS referencias.

```
class Nodo {
    int dato;
    Nodo siguiente;
}
class Cola {
    Nodo frente;
    Nodo fin;
}
```

¿Por qué dos referencias? — con solo frente, agregar al final obligaría a recorrer toda la cola entera

Con fin — alcanza con dos líneas: fin.siguiente = nuevo; fin = nuevo; → enqueue es O(1)

Para quitar — frente = frente.siguiente; → dequeue también es O(1)

Mismo truco que vimos con la lista: guardar una referencia extra (fin) convierte una operación O(n) en O(1).

Cola con array: el problema y la solución circular

También se puede implementar con un array, pero aparece un problema particular después de varios dequeue.

Después de algunos dequeue, quedan posiciones libres AL PRINCIPIO del array:



fin ya está en el último índice, pero hay 2 lugares libres que no se aprovechan.

SOLUCIÓN

Array circular: pensamos el final conectado con el principio. Cuando fin llega al último índice, el siguiente lugar puede ser el índice 0.




```
siguiente_posición = (fin + 1) % capacidad
```

Cola en Java: Queue y ArrayDeque

Igual que con la pila, Java ya trae una implementación lista para usar.

```
Queue<Integer> cola = new ArrayDeque<>();

cola.offer(10);    // agregar (enqueue)
cola.peek();       // consultar el frente
cola.poll();       // quitar y devolver (dequeue)
cola.isEmpty();    // ¿está vacía?
```

offer / poll — son los nombres que usa la interfaz Queue en vez de enqueue/dequeue, pero hacen exactamente lo mismo

La estructura lógica — sigue siendo FIFO; lo que cambia es que ya no programamos nodos, frente ni fin a mano

Nuestro trabajo pasa a ser elegir correctamente la estructura que necesitamos, no reinventarla cada vez.

¿Dónde aparecen las colas? (y la bicola)

El patrón general detrás de toda cola es: llegan → esperan → se procesan, respetando el orden de llegada.

Tareas de impresión, mensajes, solicitudes a un servidor — cualquier "fila de espera" real

Procesos pendientes y atención de clientes — el orden de llegada define el orden de atención

Variante — la bicola (Deque) permite insertar y eliminar por AMBOS extremos, no solo uno; es una extensión natural de la idea de cola

Árboles



De lo lineal a la jerarquía

Lista, pila y cola comparten algo: siempre podemos imaginarlas como "uno detrás de otro". Pero no todos los problemas tienen esa forma.

Pensemos en un sistema de archivos:

- 📁 **Documentos**
  - 📁 Universidad
    - 📁 POO
    - 📁 Estructuras
  - 📁 Personal
    - 📁 Fotos

Un elemento puede tener varios elementos debajo. Ya no es una secuencia.

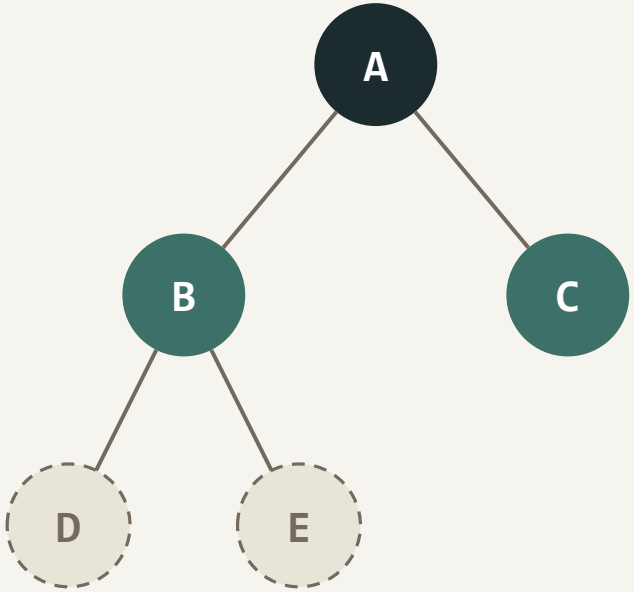
JERARQUÍA

Esa es exactamente la idea que organiza un árbol.

Aparecen relaciones nuevas que la lista nunca necesitó: padre, hijo, hermano, descendiente.

Aparece el árbol: términos básicos

Un árbol organiza elementos mediante relaciones jerárquicas, no secuenciales.



Raíz — A: el único nodo sin padre, el punto de partida del árbol

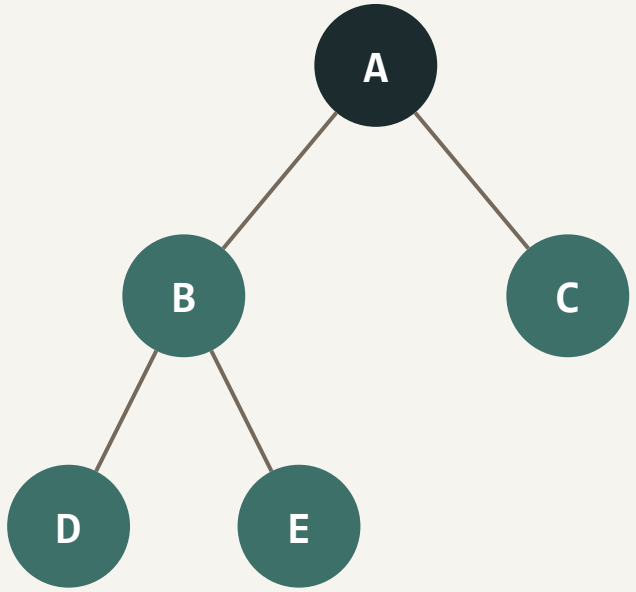
Hijos — B y C son hijos de A; D y E son hijos de B

Hermanos — B y C (comparten el mismo padre); también D y E

Hojas — D y E: nodos sin ningún hijo

Nivel, profundidad y altura

Podemos medir la posición de cada nodo dentro del árbol.



nivel 0

nivel 1

nivel 2

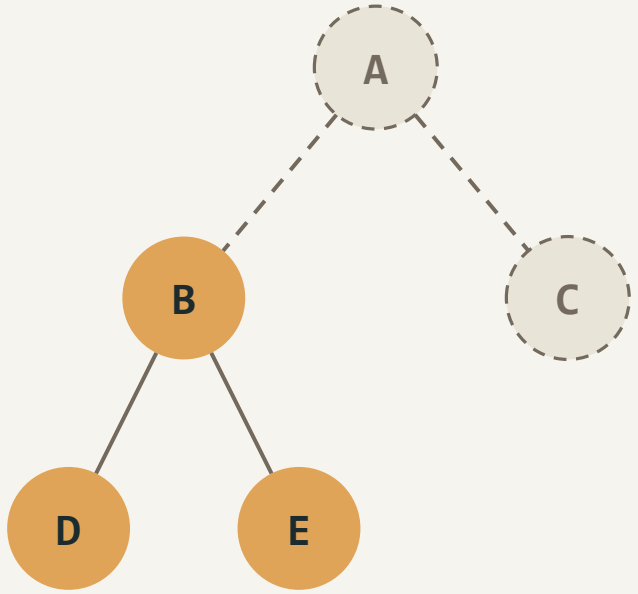
Profundidad — qué tan lejos está un nodo de la raíz (la raíz tiene profundidad 0)

Altura — se relaciona con la distancia desde un nodo hasta su descendiente más lejano

Nivel — agrupa a los nodos que están a la misma profundidad (A solo; B y C; D y E)

Subárboles

Cada nodo puede considerarse la raíz de su propio árbol más chico.



el subárbol de B

El subárbol de B — está formado por B y todo lo que cuelga de él (en este caso, D y E)

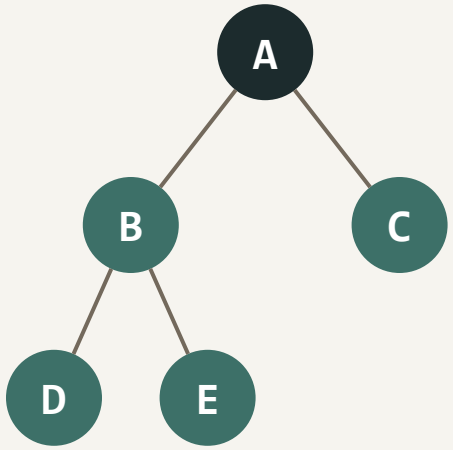
Por qué importa — muchas operaciones sobre árboles se definen pensando "resolvé esto para el subárbol izquierdo, resolvé esto para el derecho, y combinalos"

Esa idea — es exactamente la que hace que las operaciones sobre árboles se expresen de forma natural en forma recursiva

Árbol binario y árbol binario completo

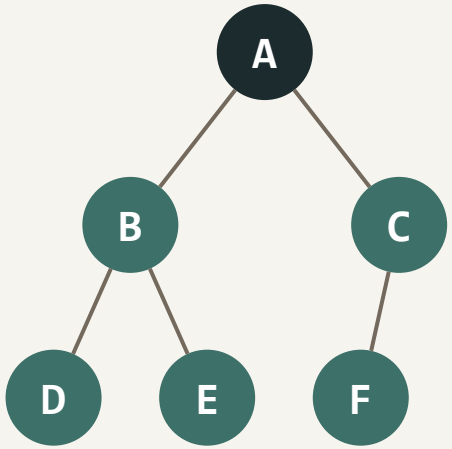
Dos propiedades distintas que suelen confundirse — vale la pena separarlas bien.

Árbol binario



Cada nodo tiene como máximo 2 hijos: izquierdo y derecho (nunca 3).

Árbol binario COMPLETO



Los niveles se ocupan en orden, dejando a lo sumo el último nivel parcialmente lleno.

No son lo mismo: todo árbol binario completo es binario, pero no todo árbol binario es completo (el de la izquierda no lo es).

Representar un árbol en Java

Volvemos a la idea del nodo — pero ahora necesita DOS referencias en vez de una.

```
class Nodo {
    int dato;
    Nodo izquierdo;
    Nodo derecho;
}
```

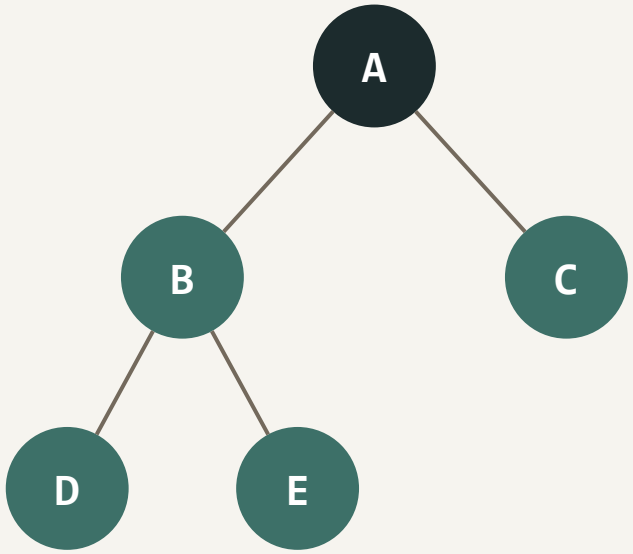
izquierdo y derecho — cada uno puede ser otro Nodo, o null si ese hijo no existe

La estructura — vuelve a construirse mediante referencias, igual que la lista — pero ahora hay dos caminos posibles desde cada nodo, no uno solo

Esta pequeña diferencia (una referencia más) es lo único que separa conceptualmente a una lista enlazada de un árbol binario.

Recorrer un árbol: tres caminos posibles

En una lista había un solo recorrido posible. En un árbol, el mismo conjunto de nodos admite varios órdenes de visita.



Preorden
`raíz → izquierdo → derecho`

Inorden
`izquierdo → raíz → derecho`

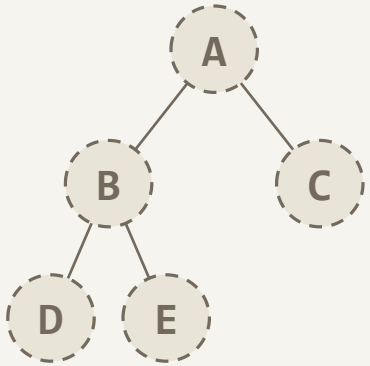
Postorden
`izquierdo → derecho → raíz`

Los tres recorridos, lado a lado

Es el mismo árbol las tres veces — lo único que cambia es el orden en que visitamos los nodos.

Preorden

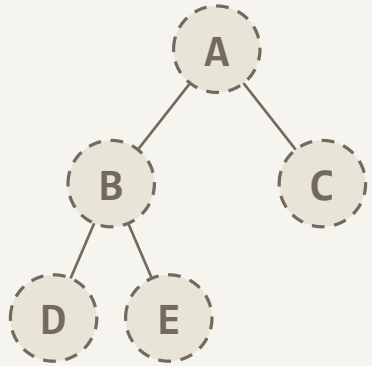
raíz · izq · der



A B D E C

Inorden

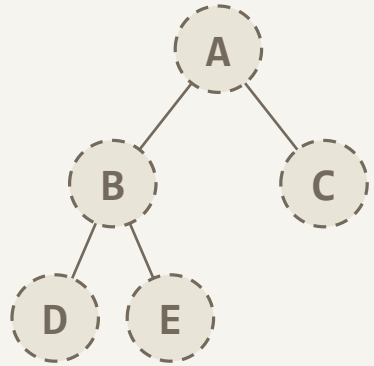
izq · raíz · der



D B E A C

Postorden

izq · der · raíz

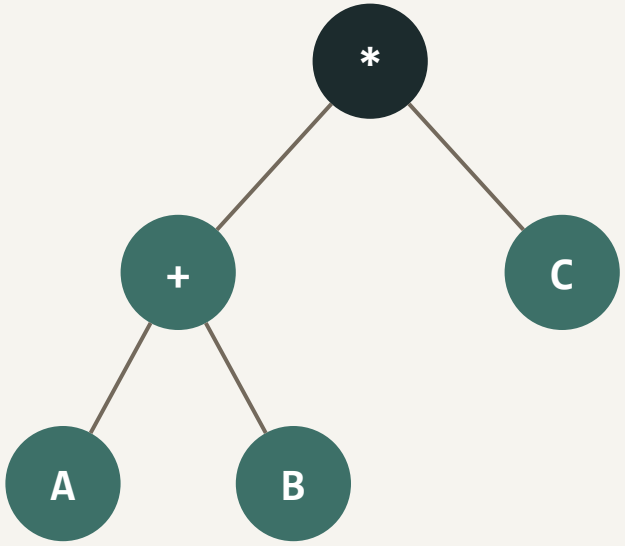


D E B C A

No son tres árboles diferentes: es el mismo árbol, recorrido siguiendo tres reglas distintas.

¿Por qué importa el orden? Un árbol de expresión

Los árboles no solo almacenan datos — también pueden representar relaciones y estructuras de información, como una expresión matemática.



Inorden de este árbol:

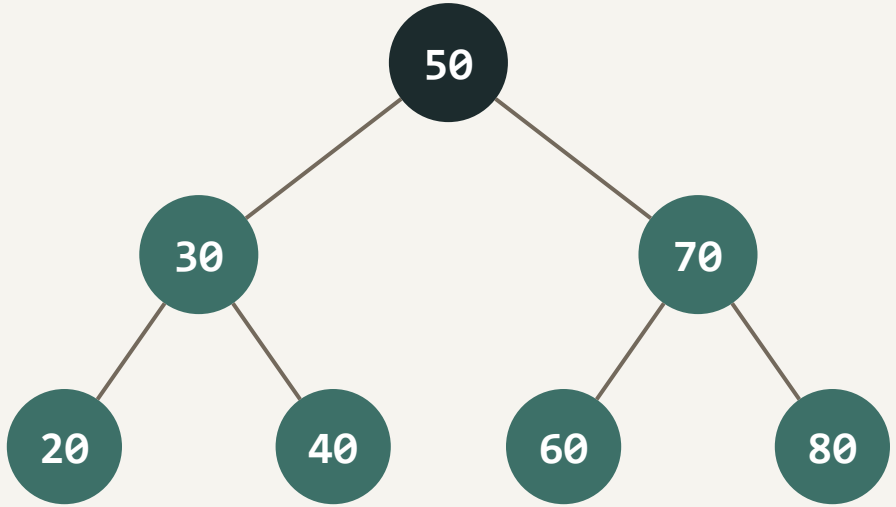
`(A + B) * C`

El recorrido elegido determina qué representación obtenemos de la misma información — no es un detalle menor, cambia el resultado.

Árbol binario de búsqueda (ABB): la regla de orden

Agregamos una única regla nueva, que vale para TODOS los nodos del árbol:

izquierda < nodo < derecha

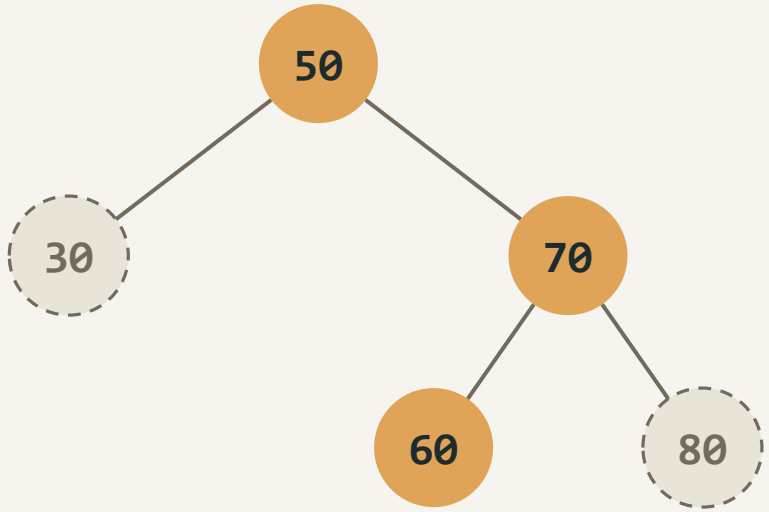


Los valores menores
quedan siempre a la izquierda de cada nodo.
Los valores mayores

Esta regla es la que permite usar al árbol para BUSCAR de forma eficiente — próxima diapositiva.

Buscar en un ABB

Queremos encontrar 60. En cada nodo, comparamos y descartamos una rama entera.



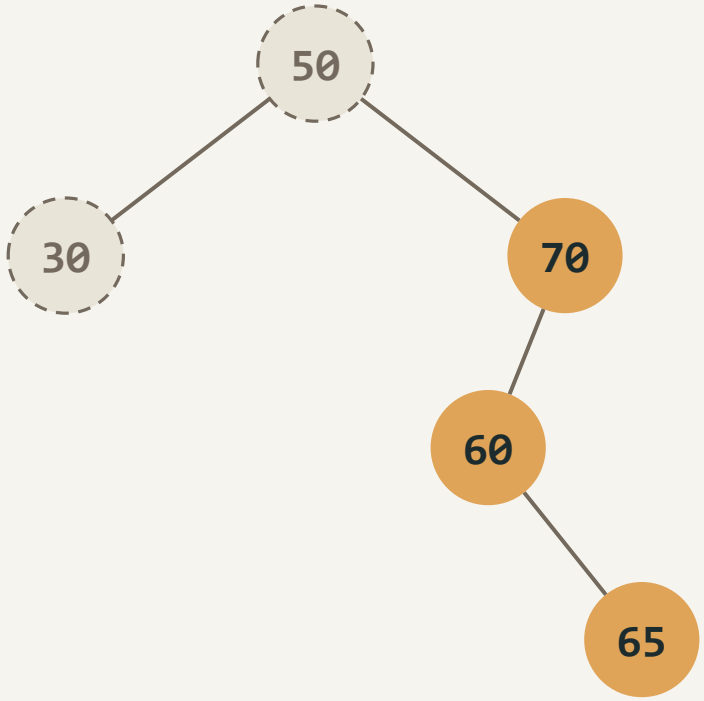
1. 60 > 50 → descartamos todo el subárbol izquierdo (30) y vamos a la derecha
2. 60 < 70 → descartamos el subárbol derecho de 70 (80) y vamos a la izquierda
3. 60 == 60 → encontrado

No hace falta recorrer todos los nodos si el árbol está bien organizado — cada paso descarta la mitad de lo que queda.

Insertar en un ABB

Queremos insertar 65. La posición correcta surge sola de ir aplicando la regla de orden.

1. 65 > 50 → derecha
2. 65 < 70 → izquierda
3. 65 > 60 → derecha (lugar libre: se inserta ahí)



La posición del nuevo nodo no se elige: la determina por completo la regla de orden del ABB.

Eliminar en un ABB: tres casos distintos



Eliminar es la operación más delicada, porque hay que mantener la regla de orden después de sacar el nodo.

Caso 1 — el nodo es una hoja: se elimina directamente, sin ningún efecto secundario

Caso 2 — el nodo tiene un solo hijo: alcanza con conectar al padre directamente con ese hijo, saltando el nodo eliminado

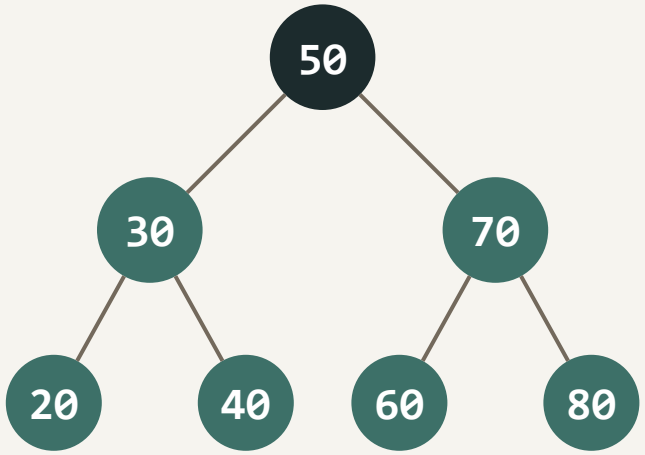
Caso 3 — el nodo tiene dos hijos: hay que reorganizar — típicamente se reemplaza el valor por el menor del subárbol derecho (o el mayor del izquierdo), y se elimina ese reemplazante de su lugar original

El caso 3 es el único realmente complejo — vale la pena dedicarle un ejemplo aparte en la práctica.

La forma del árbol importa

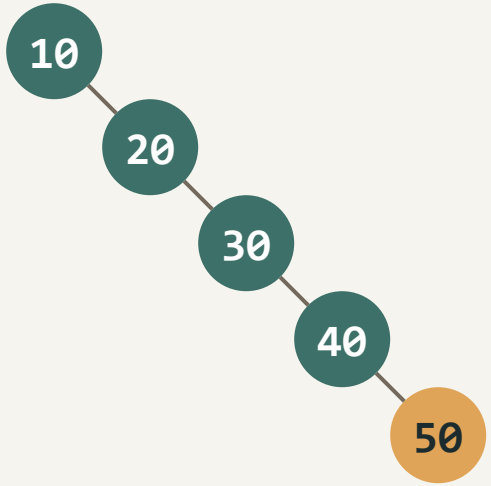
La misma cantidad de datos puede dar lugar a árboles muy distintos, según el orden de inserción.

Bien balanceado



Búsqueda: O(log n) — cada paso descarta la mitad del árbol

Insertado en orden (degenerado)



Búsqueda: O(n) — el árbol degeneró en una lista disfrazada

La eficiencia de un ABB depende de qué tan balanceado esté — no es una propiedad automática, depende del orden de inserción.

Java no tiene un "árbol genérico" para esto



A diferencia de la pila y la cola, acá no hay un atajo de la biblioteca estándar que reemplace lo que construimos.

Java sí tiene — muchas colecciones (TreeMap, TreeSet usan árboles internamente), pero no una clase ArbolBinario que represente directamente lo que construimos hoy

Por eso — para entender esta estructura, conviene trabajar con el Nodo propio (izquierdo/derecho) y programar las operaciones a mano

Operaciones a implementar — insertar(), buscar(), eliminar(), y los tres recorridos: preorden(), inorden(), postorden()

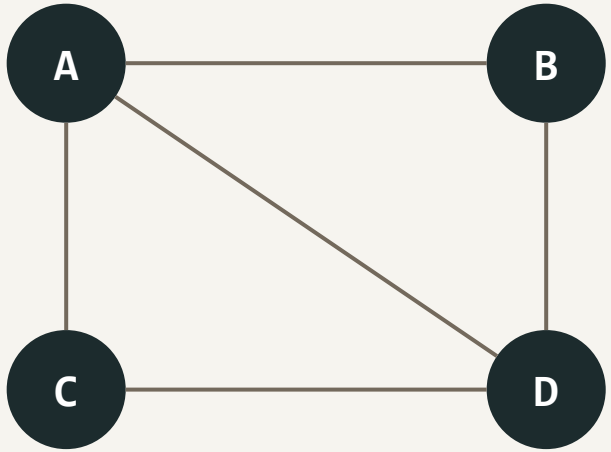
Acá la implementación propia tiene un valor especialmente importante: no hay versión "de fábrica" para comparar.

Grafos



¿Y si la relación deja de ser jerárquica?

Un árbol tiene raíz, padres e hijos. Pero pensemos en una red de ciudades conectadas por rutas:



A se relaciona con B, con C y con D — no tiene un único "camino hacia abajo"

No existe necesariamente — una raíz, ni una noción fija de padre/hijo

Necesitamos — una estructura más general, donde cualquier elemento pueda relacionarse con cualquier otro

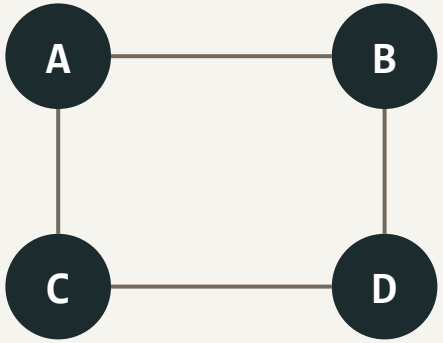
GRAFO

¿Qué es un grafo?

```
G = (V, A)
V = {A, B, C, D}
A = {(A,B), (A,C), (B,D), (C,D)}
```

Vértices (V) — representan los elementos (en el ejemplo, cuatro: A, B, C, D)

Aristas o arcos (A) — representan las relaciones entre pares de vértices



Grafo dirigido vs. no dirigido

La arista puede o no tener un sentido definido — y eso cambia por completo qué relaciones son válidas.

No dirigido



A ↔ B (la relación va en los dos sentidos)

Dirigido

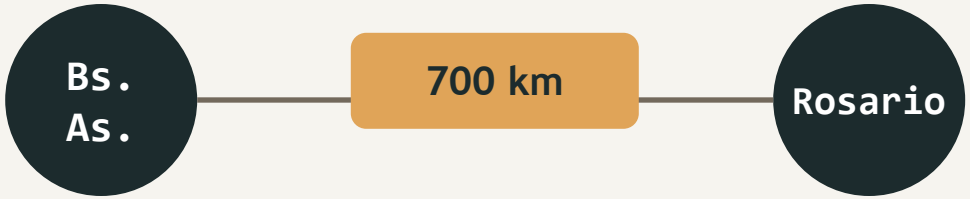


A → B (no implica necesariamente B → A)

Ejemplo de cuándo usar cada uno: "amistad en una red social" suele ser no dirigido; "sigue a" en Facebook/IG es dirigido — seguir a alguien no implica que te siga de vuelta.

Grafos valorados (con peso)

Una relación también puede tener un valor numérico asociado — no solo "existe o no existe".



El valor (o peso) — puede representar distancia, costo, tiempo o capacidad, según qué estemos modelando

Notación habitual — A ──peso── B, o simplemente A — 10 — B

Por qué importa — una vez que las aristas tienen peso, preguntas como "¿cuál es el camino más corto?" empiezan a tener sentido (y algoritmo propio, que no vemos en detalle hoy)

Conceptos que aparecen en los grafos

Un vocabulario chico, pero que se usa todo el tiempo al hablar de grafos.

Adyacentes — dos vértices lo son cuando existe una arista que los relaciona directamente

Camino — una secuencia de vértices conectados, uno detrás de otro, por aristas

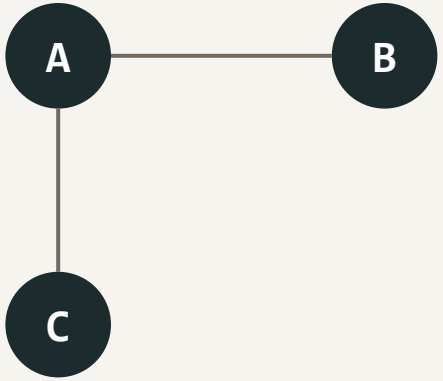
Ciclo — aparece cuando un camino nos permite volver al vértice de partida sin repetir aristas

Conexo — un grafo es conexo cuando existe al menos un camino entre cualquier par de sus vértices

Estos cuatro conceptos alcanzan para describir la estructura de casi cualquier grafo que aparezca en la práctica.

Representar un grafo: matriz de adyacencia

Una forma de guardar las relaciones: una tabla donde cada celda indica si dos vértices están conectados.



	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

Un 1 indica que existe relación entre ese par de vértices; un 0 indica que no. Útil cuando el grafo es denso (muchas conexiones).

Representar un grafo: listas de adyacencia

Otra opción: guardar, para cada vértice, la lista de vértices con los que está conectado.

```
A: [B, C]
B: [A]
C: [A]
```

Útil cuando el grafo es disperso — pocas conexiones en relación a la cantidad de vértices (no desperdicia espacio en ceros, como sí hace la matriz)

Matriz vs. listas — no hay una representación universalmente mejor: grafo denso → matriz; grafo disperso → listas

La elección de representación — es parte del diseño de la estructura, igual que elegimos entre nodos o array para la pila

Recorrer un grafo: el problema de los ciclos

Recorrer un grafo significa visitar los vértices alcanzables desde un vértice de partida.

La lista y el árbol — no tenían ciclos: siempre se llegaba a null o a una hoja, el recorrido terminaba solo

El grafo sí puede tener ciclos — A → B → C → A es perfectamente válido, y un recorrido ingenuo entraría en un bucle infinito

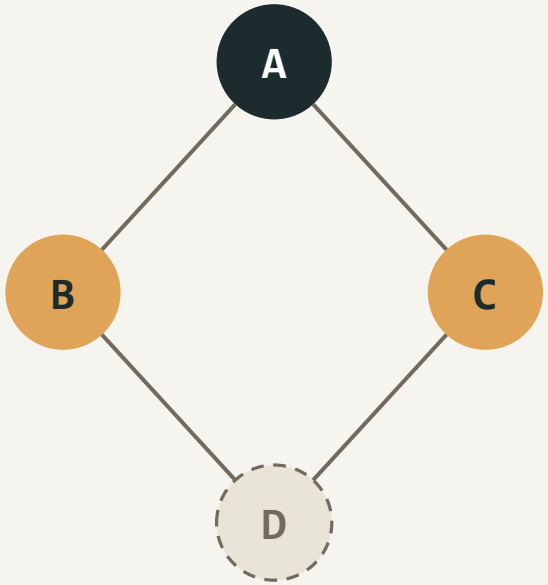
La solución — llevar un registro de qué vértices ya visitamos, y no volver a entrar a los que ya están marcados

Con esa única precaución (marcar visitados), aparecen dos estrategias de recorrido clásicas: BFS y DFS.

BFS — Recorrido en anchura

Breadth First Search

La idea: primero visitamos todos los vecinos directos, y recién después los vecinos de esos vecinos.



BFS usa una COLA.

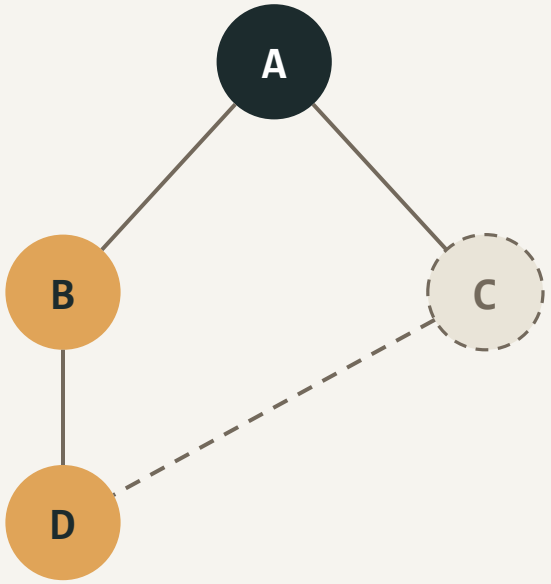
Encolamos A; lo sacamos y encolamos sus vecinos (B, C); los sacamos de a uno y encolamos SUS vecinos no visitados (D).

BFS → COLA → FIFO

DFS — Recorrido en profundidad

Depth First Search

La idea: seguir un camino hasta el final, y recién ahí retroceder para probar otro camino.



DFS usa una PILA.

Desde A profundizamos por B hasta D antes de siquiera mirar a C. Se puede implementar con una pila explícita, o directamente con recursividad.

DFS → PILA → LIFO

BFS vs. DFS

	BFS	DFS
Estructura que usa	Cola	Pila
Regla	FIFO	LIFO
Estrategia	Avanza por niveles	Profundiza por caminos antes de retroceder
Implementación típica	Cola explícita	Pila explícita, o recursividad

Una mirada completa a la unidad

Array – Lista – Pila / Cola – Árbol – Grafo

La pregunta no es "¿qué estructura sé implementar?" — es "¿qué relación existe entre los datos, y qué operaciones necesito?"

Secuencia general	LISTA
Último en entrar, primero en salir	PILA
Primero en entrar, primero en salir	COLA
Jerarquía	ÁRBOL
Relaciones generales	GRAFO

Para la práctica

Colas

- Implementar enqueue(), dequeue(), peek(), isEmpty()
- Primero con nodos propios
- Después comparar con Queue<T> / ArrayDeque<T>

Árbol

- Implementar insertar(), buscar(), eliminar()
- Implementar preorden(), inorden(), postorden()
- Probar con un árbol bien balanceado y uno degenerado

Grafo

- Representar el mismo grafo con matriz de adyacencia
- Y también con listas de adyacencia
- Comparar ventajas según densidad del grafo

Cierre de la unidad

Las estructuras de datos no son solo formas distintas de guardar información: cada una impone una forma particular de relacionar, acceder y procesar los datos.

Lista **Pila** **Cola** **Árbol** **Grafo**

Son respuestas diferentes a problemas diferentes. Y una vez que entendemos la estructura, Java nos permite elegir cuándo usar una implementación ya disponible y cuándo construir una propia.